pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Analyzing Text, Part 1

WEEK 4, DAY 2, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note

We keep every FOMC statement and have only ever read them. Before we run all 46, prove the tools on one, 2023-11-01: what figures does it state, how often does it say "rate", the word, and who voted? I can count the names myself, so your count must agree with mine. Then our own book: the broker sends bond descriptions as one line of text, and I want a table; and every CUSIP in the holdings file must have the right shape. Counts only: no reading of policy.

The statements	
Source	Board of Governors of the Federal Reserve System, each statement from its press release: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm
Retrieved	2026-10-03
Span	46, 2021-01-27 to 2026-09-16, every scheduled meeting
Reuse	Public domain, citing the Board as the source: https://www.federalreserve.gov/disclaimer.htm
Changes	White space only

The descriptions and CUSIPs come from the course's holdings file; the issuers are invented.

Five asks: the figures, the word rate, the voters, the descriptions, the CUSIPs.

17.1 One statement, five paragraphs

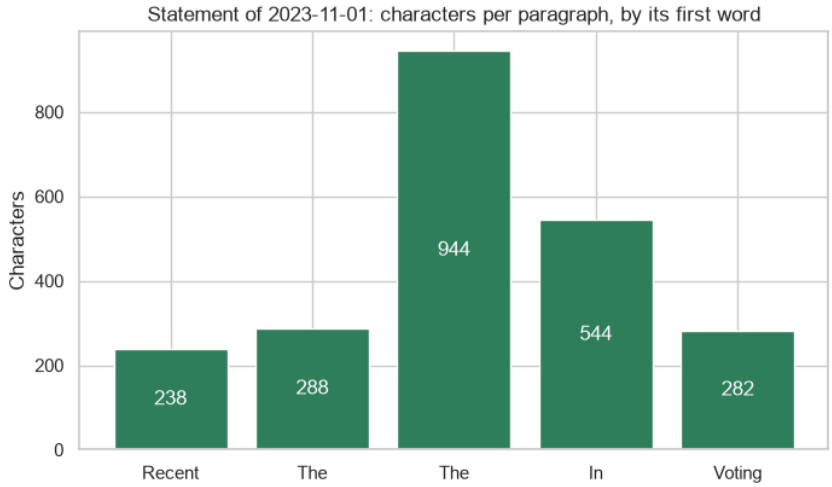
```
statements = pd.read_csv(data_folder + 'fomc_statements.csv')
print(statements.shape, statements['date'].min(), statements['date'].max())
gaps, repeats = statements.isna().sum().sum(), statements.duplicated().sum()
print(gaps, repeats, statements['text'].is_unique)
(46, 3) 2021-01-27 2026-09-16
0 0 True
```

No gaps, no repeats: a statement in twice would double every count.

```
statement = statements.loc[statements['date'] == '2023-11-01', 'text'].iloc[0]
paragraphs = statement.split('\n')
print(type(statement), len(paragraphs))
opening = paragraphs[0]
print(opening)
<class 'str'> 5
```

Recent indicators suggest that economic activity expanded at a strong pace in the third quarter. Job gains have moderated since earlier in the year but remain strong, and the unemployment rate has remained low. Inflation remains elevated.

A "str", like Day 1's tickers, only longer. '\n' is Day 2's line break, the one that broke a typed path.



The third paragraph, 944 characters, is the longest; the last, "Voting", lists names.

Figures in the third, voters in the last. Start with the first.

Source: Board of Governors of the Federal Reserve System, each statement from its press release: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm

17.2 How long, is it there, where, how many words

```
print(len(opening), len(statement))
print('inflation' in opening, 'inflation' in opening.lower())
238 2300
False True
```

238 and 2,300 characters. "False", then "True": the paragraph says "Inflation", Day 1's "'energy' in issuer" trap.

```
position = opening.find('Inflation')
print(position)
print(opening[:6], opening[position:position + 9], opening[-9:])
211
Recent Inflation elevated.
```

211: Python counts from 0, as Day 1's positions picture showed, so it is the 212th character.

```
words = opening.split()
print(len(words), words[-3:])
37 ['Inflation', 'remains', 'elevated.']
```

37 words, and 'elevated.' keeps its full stop: "split()" cuts at spaces and nothing else.

```
print(' '.join(words[:5]))
print('-'.join(words[:5]))
Recent indicators suggest that economic
Recent-indicators-suggest-that-economic
```

Day 1's string methods work the same on a paragraph.

```
pieces = opening.split('.')
for piece in pieces:
    print(repr(piece))
'Recent indicators suggest that economic activity expanded at a strong pace in the third quarter'
' Job gains have moderated since earlier in the year but remain strong, and the unemployment rate has remained low'
' Inflation remains elevated'
''
```

Two pieces start with a space, and the last is empty: what follows the final full stop.

Sentences, and the same jobs in Excel

```
sentences = [piece.strip() for piece in pieces if piece.strip() != '']
print(len(sentences), sentences[2])
3 Inflation remains elevated
```

3 sentences. A full stop is a rough cut: the next paragraph's "U.S." would fall apart.

Excel side by side. The paragraph in A2

Job	Python	Excel	Result
Where a word starts	`opening.find('Inflation')`	=FIND("Inflation", A2), then =MID(A2, 212, 9)	211 against 212: Excel counts from 1
Is it there	`'inflation' in opening`	=ISNUMBER(FIND("inflation", A2)); SEARCH ignores capitals	FALSE; TRUE with SEARCH
Count a piece	`opening.count('rate')`	=(LEN(A2)-LEN(SUBSTITUTE(A2, "rate", "")))/LEN("rate")	
Count the words	`len(opening.split())`	=COUNTA(TEXTSPLIT(A2, " "));	37
		any version: =LEN(TRIM(A2))-LEN(SUBSTITUTE(TRIM(A2), " ", ""))+1	

"TEXTSPLIT" is in Microsoft 365 and Excel 2024; Text to Columns splits by hand.

Every tool here looks for one exact piece of text. A figure changes its digits every meeting.

17.3 A shape, not a text: the regular expression

"2 percent" and "5-1/4 to 5-1/2 percent" share a shape: digits, then percent. A regular expression, or regex, writes a shape down. Python's "re" module has "re.findall()" for every match, "re.search()" for the first, and "re.sub()" to replace.

> *The Committee seeks to achieve maximum employment and inflation at the rate of 2 percent over the longer run. In support of these goals, the Committee decided to maintain the target range for the federal funds rate at 5-1/4 to 5-1/2 percent.*

```
import re
policy = paragraphs[2]
policy_sentences = policy.split('. ')
print(len(policy), len(policy_sentences))
print(policy_sentences[0] + '.')
print(policy_sentences[1] + '.')
944 6
```

944 characters in 6 sentences.

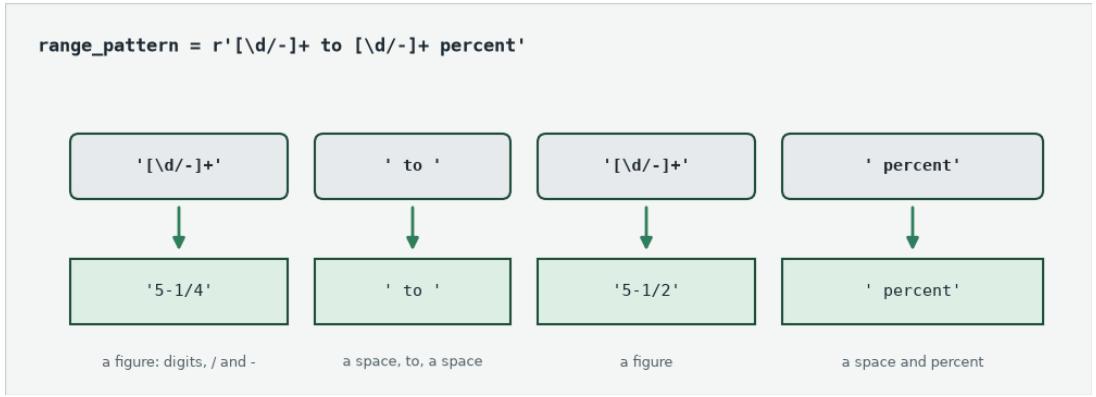
```
print(re.findall('percent', policy))
print(re.findall(r'\d', policy))
print(re.findall(r'\d+', policy))
['percent', 'percent', 'percent', 'percent']
['2', '5', '1', '4', '5', '1', '2', '2', '2']
['2', '5', '1', '4', '5', '1', '2', '2', '2']
```

9 digits, the same list both ways: no figure here has two digits in a row.

The "r" makes a raw string, where a backslash stays a backslash: write every pattern as "r'...'".

A class of characters, the range, and its two ends

```
range_pattern = r'[\d/-]+ to [\d/-]+ percent'
print(re.findall(range_pattern, policy))
['5-1/4 to 5-1/2 percent']
```




```
print(re.findall(r'([\d/-]+) to ([\d/-]+) percent', policy))
match = re.search(r'([\d/-]+) to ([\d/-]+) percent', policy)
print(match.group(0), '|', match.group(1), '|', match.group(2), '|', match.start())
[('5-1/4', '5-1/2')]
5-1/4 to 5-1/2 percent | 5-1/4 | 5-1/2 | 218
```

"group(0)" is the whole match, "group(1)" and "group(2)" the bracketed pieces, starting at position 218. The ends are text: "5-1/4" becomes 5.25 only by arithmetic.

Piece	Means
[a-z]	one lower-case letter: a hyphen between two is a range
[\d/-]	a digit, a slash, or a hyphen, plain when last
[^abc]	one character that is not a, b, or c

```
print(re.sub(range_pattern, '[RANGE]', policy_sentences[1]))
In support of these goals, the Committee decided to maintain the target range for the federal funds rate at [RANGE]
```

"re.sub()" is "replace()" with a pattern: it marks any range in this style, whatever its digits.

The figures: 2 percent three times, and a range of 5-1/4 to 5-1/2 percent.

17.4 No match, and how much a pattern takes

```
re.search(r'\d+ basis points', policy).group(0)
AttributeError: 'NoneType' object has no attribute 'group'
```

Read the last line, as on Day 3: with no match, "search()" returns "None", Python's value for nothing, and "None" has no "group()".

```
match = re.search(r'\d+ basis points', policy)
if match is None:
    print('No figure in basis points in this paragraph.')
else:
    print(match.group(0))
No figure in basis points in this paragraph.
```

Piece	Means
\w	a letter, a digit, or an underscore
\b	the edge of a word
*, ?, {4}	none or more; none or one; exactly 4
\s, \S	a space, a tab, or a line break; any character that is not one
.	any character but a line break; \. is a full stop

"\b" matches no character, only the place where a word starts or ends.

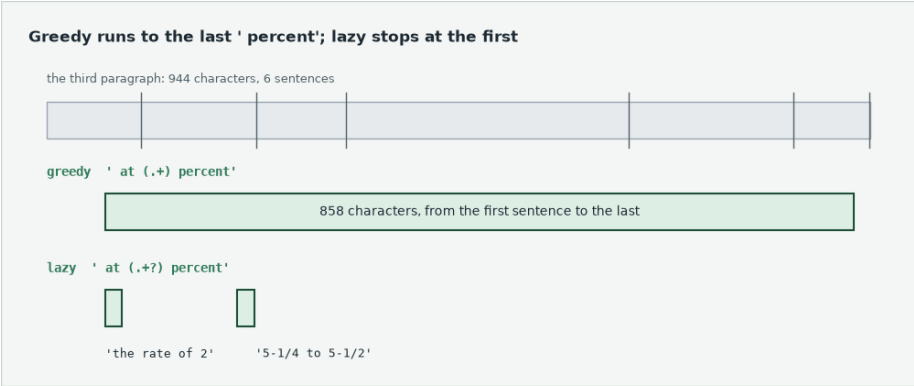
"+" and "*" are greedy: they take as much text as they can. A "?" straight after one makes it lazy: as little as it can.

```
greedy = re.search(r' at (.+) percent', policy).group(1)
lazy = re.search(r' at (.+?) percent', policy).group(1)
print(len(greedy), 'of', len(policy), 'characters')
print(lazy)
print(re.findall(r' at (.+?) percent', policy))
858 of 944 characters
the rate of 2
['the rate of 2', '5-1/4 to 5-1/2']
```

Greedy took 858 of the 944 characters, from the first sentence to the last; lazy took 13.

The range pattern cannot run on like this: a letter is not in its brackets.

Greedy and lazy, a story, and Excel



Cloudflare, July 2, 2019

On July 2, 2019, Cloudflare deployed a new rule in its web application firewall. The critical part of its regular expression was ".*(?:.*=.*)": three greedy runs around one equals sign. The matcher tried so many ways to fit them that the CPUs serving web traffic ran at nearly 100 percent, and the service went down for 27 minutes.

Source: https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/

Excel side by side. The third paragraph in A2

Job	Python	Excel
The first match	`re.search(pattern, text).group(0)`	=REGEXEXTRACT(A2, pattern): 5-1/4 to 5-1/2 percent
Between two markers	`re.search(r' at (.+?) percent', text).group(1)`	=TEXTBEFORE(TEXTAFTER(A2, " at "), " percent"): the rate of 2, the lazy answer
Cells holding a piece	`count() on each word`	=COUNTIF(B2:B38, "*rate*") on the 37 words

"REGEXEXTRACT" is in Microsoft 365 only; "TEXTBEFORE" and "TEXTAFTER" are in Microsoft 365 and Excel 2024; the "*" wildcard works in every version. Flash Fill (Ctrl+E) copies a typed pattern, as values, not a formula.

17.5 Who voted: a pattern tied to an outside count

```
print(re.findall(r'^\w+', statement, re.MULTILINE))
print(re.findall(r'\w+\.$', statement, re.MULTILINE))
['Recent', 'The', 'The', 'In', 'Voting']
['elevated.', 'risks.', 'objective.', 'developments.', 'Waller.']
```

"^" and "$" are anchors: a pattern tied to the start or the end cannot match in the middle.

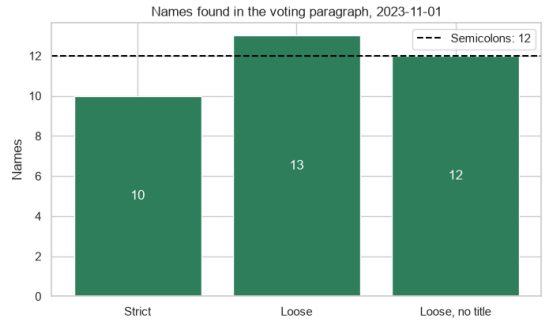
```
voting = paragraphs[-1]
print(voting)
print('By semicolons:', voting.count(';') + 1)
strict = re.findall(r'[A-Z][a-z]+ [A-Z]\. [A-Z][a-z]+', voting)
print('By the pattern:', len(strict))
By semicolons: 12
By the pattern: 10
```

Voting for the monetary policy action were Jerome H. Powell, Chair; John C. Williams, Vice Chair; Michael S. Barr; Michelle W. Bowman; Lisa D. Cook; Austan D. Goolsbee; Patrick Harker; Philip N. Jefferson; Neel Kashkari; Adriana D. Kugler; Lorie K. Logan; and Christopher J. Waller.

The pattern found 10 of the 12: it matched too little, as Patrick Harker and Neel Kashkari have no middle initial, and it insists on one.

```
loose = re.findall(r'[A-Z][a-z]+ (?:[A-Z]\. )?[A-Z][a-z]+', voting)
print(len(loose), loose[:3])
names = [name for name in loose if name != 'Vice Chair']
print(len(names))
13 ['Jerome H. Powell', 'John C. Williams', 'Vice Chair']
12
```

13: "Vice Chair" starts each word with a capital, all the loose pattern asks; without it, 12. Tighten a pattern and real names drop out; loosen it and false ones come in.



The count from the semicolons decided which way each pattern was wrong.

17.6 The broker's descriptions, read into a table

```
descriptions = ['KRIX 7.875 08/26/2032', 'BEZE 7 06/18/2032', 'BEZH 8.125 04/17/2029',
                'BEZL 5.125 06/03/2030', 'LORE 7.25 03/08/2030', 'OSTV  6.5 04/01/2033',
                'SYLX 3.625 09/03/2055', 'VYRN 7.375 12/12/2037']
first_pattern = r'([A-Z]+) (\d+\.\d+) (\d{2}/\d{2}/\d{4})'
print(re.fullmatch(first_pattern, descriptions[0]).groups())
('KRIX', '7.875', '08/26/2032')
```

Three pieces, all text, the coupon included.

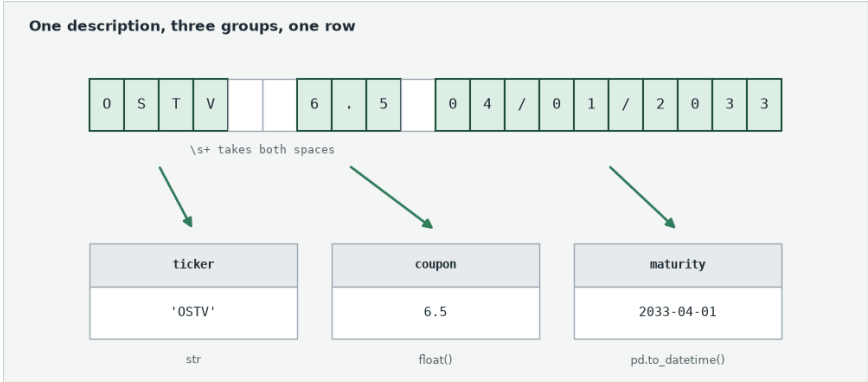
```
description_pattern = r'([A-Z]+)\s+(\d+(?:\.\d+)?)\s+(\d{2}/\d{2}/\d{4})'
rows = []
for text in descriptions:
    match = re.fullmatch(description_pattern, text)
    assert match is not None, 'cannot read: ' + text
    ticker, coupon, maturity = match.groups()
    rows.append({'ticker': ticker, 'coupon': float(coupon), 'maturity': maturity})
bonds = pd.DataFrame(rows)
```

Excel side by side

"=TEXTSPLIT(TRIM(A2), " ")" in B2 spills the three pieces into B2:D2, "TRIM" squeezing the double space; "=VALUE(C2)" makes the coupon a number. In Microsoft 365, "=REGEXEXTRACT(A2, pattern, 2)" gives one cell per group.

```
bonds['maturity'] = pd.to_datetime(bonds['maturity'], format='%m/%d/%Y')
print(bonds)
print(round(bonds['coupon'].mean(), 3), bonds['maturity'].max().date())
  ticker  coupon   maturity
0   KRIX   7.875 2032-08-26
1   BEZE   7.000 2032-06-18
2   BEZH   8.125 2029-04-17
3   BEZL   5.125 2030-06-03
4   LORE   7.250 2030-03-08
5   OSTV   6.500 2033-04-01
6   SYLX   3.625 2055-09-03
7   VYRN   7.375 2037-12-12
6.609 2055-09-03
```

8 of 8: mean coupon 6.609 percent, latest maturity 2055-09-03.



A whole value: a date, then every CUSIP in the book

Anchors at both ends test a whole value, such as a date.

```
date_pattern = r'^\d{4}-\d{2}-\d{2}$'
for value in ['2023-11-01', '2023-11-1', '11/01/2023']:
    print(value, re.search(date_pattern, value) is not None)
url = statements.loc[statements['date'] == '2023-11-01', 'url'].iloc[0]
print(re.search(r'(\d{4})(\d{2})(\d{2})', url).groups())
2023-11-01 True
2023-11-1 False
11/01/2023 False
('2023', '11', '01')
```

The address agrees.

```
repaired = re.sub(r'[^0-9A-Z]', '', '99080-CAE8 ')
print(repr(repaired), re.fullmatch(cusip_shape, repaired) is not None)
print(re.sub(r'[^0-9A-Z]', '', '99080cae8'))
'99080CAE8' True
990808
```

The hyphen and the space go, and the copy passes; the lower-case copy shrinks to "990808", the wrong repair.

200 of 200 have the shape; the ninth character, a check digit, is a test for another day.

A CUSIP is Day 1's nine boxes, here each a digit or a capital.

```
cusips = pd.read_csv(data_folder + 'holdings.csv', dtype={'cusip': str})['cusip']
cusip_shape = r'[0-9A-Z]{9}'
has_shape = cusips.str.fullmatch(cusip_shape)
print(len(cusips), has_shape.sum())
assert has_shape.all(), 'a CUSIP without the shape'
200 200
```

```
for value in [cusips.iloc[0], '99080cae8', '99080CAE', '99080CAE8 ', '99080-CAE8']:
    print(repr(value), re.fullmatch(cusip_shape, value) is not None)
'99080CAE8' True
'99080cae8' False
'99080CAE' False
'99080CAE8 ' False
'99080-CAE8' False
```

Each copy fails, and "repr()" shows the trailing space the eye misses.

Excel side by side

"=REGEXTEST(B2, "^[0-9A-Z]{9}$")" in Microsoft 365, anchored because it searches; in every version, "=LEN(B2)=9" and "=EXACT(B2, UPPER(B2))" test the length and the capitals.

Predict first: decide, then run each in the notebook

Predict 1

In the paragraph printed in 17.1, the word rate appears once: "the unemployment rate". What does "opening.count('rate')" return? Write down a number.

```
print(opening.count('rate'))
print(opening.count('remain'))
```

Predict 2

Read to its end, the paragraph says "2 percent" three times. How many matches does "re.findall(r'\d+ percent', policy)" return? Write down a number.

```
print(re.findall(r'\d+ percent', policy))
```

Predict 3

"statement.count('rate')" returns 4 for the whole statement. What does "re.findall(r'\brate\b', statement)" return? A) 4 matches, as "count()" does. B) 3 matches. C) No matches: the backslashes spoil the pattern.

```
print(statement.count('rate'))
print(re.findall(r'\w*rate\w*', statement))
print(re.findall(r'\brate\b', statement))
```

Predict 4

Look at the eight descriptions in the cell above. How many will "first_pattern" read? Write down a number.

```
not_read = [text for text in descriptions if re.fullmatch(first_pattern, text) is None]
print(len(descriptions) - len(not_read), not_read)
```

The note, answered

The note asked	Answer
The figures	2 percent three times, and 5-1/4 to 5-1/2 percent
Who voted	12 names, as the semicolons say
The descriptions as a table	8 of 8, mean coupon 6.609 percent
Every CUSIP's shape	200 of 200

By the end of this notebook, you can
Measure, search, count, split, join, and strip text.
Build a regex from "\d", "\w", "\s", "\b", classes, "+", "*", "?", anchors, and groups.
Use "re.findall()", "re.search()", and "re.sub()", and tie a pattern to an outside count.
Tell greedy from lazy, and test a whole value with "re.fullmatch()".
Read bond descriptions into a table and test every CUSIP's shape.

Part 2 cleans the text, counts its words, and runs the same work on all 46 statements.